

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Практическая работа №5

По дисциплине Информационная безопасность

Тема работы Криптографические алгоритмы

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Пикуль А.С.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ШИФР ЦЕЗАРЯ	4
1.1 Реализация шифра Цезаря.....	4
1.2 Лавинность подстановки и количество ключей	5
1.3 Взлом шифра Цезаря частотным анализом	5
1.4 Почему частотный анализ работает.....	6
2 ШИФР ВИЖЕНЕРА	7
2.1 Реализация шифра Виженера	7
2.2 Отличие от шифра Цезаря.....	8
2.3 Оценка длины ключа через индекс совпадений.....	8
2.4 Полный взлом шифра Виженера	10
ЗАКЛЮЧЕНИЕ	11

ВВЕДЕНИЕ

Цель работы: реализовать на Python классические шифры подстановки — шифр Цезаря и шифр Виженера, — научиться вскрывать их частотным анализом без знания ключа и на собственном опыте понять принцип Керкгоффса: почему стойкость криптосистемы должна опираться на секретность ключа, а не алгоритма.

Задачи:

- 1) реализовать функции шифрования и дешифрования для шифра Цезаря и Виженера;
- 2) продемонстрировать лавинность подстановки и объяснить количество возможных ключей;
- 3) реализовать взлом шифра Цезаря перебором с использованием метода хи-квадрат;
- 4) реализовать оценку длины ключа шифра Виженера через индекс совпадений;
- 5) реализовать полный взлом шифра Виженера с разбиением на потоки и частотным анализом каждого потока.

1 ШИФР ЦЕЗАРЯ

1.1 Реализация шифра Цезаря

В файле `student_caesar.py` реализованы функции `caesar_encrypt` и `caesar_decrypt`. На рисунке 1 приведён код реализованных функций, а на рисунке 2 — результат проверки: шифрование с ключом 3 и последующая расшифровка, которая полностью совпадает с исходным сообщением.



```
17 def caesar_encrypt(text: str, key: int) -> str:
18     """Зашифровывает текст шифром Цезаря со сдвигом key"""
19     text = normalize(text)
20     result = []
21     for char in text:
22         old_index = ALPHABET_DICT[char]
23         new_index = (old_index + key) % N
24         result.append(ALPHABET[new_index])
25     return "".join(result)
26
27
28 def caesar_decrypt(cipher: str, key: int) -> str:
29     """Расшифровывает текст шифра Цезаря со сдвигом key"""
30     cipher = normalize(cipher)
31     result = []
32     for char in cipher:
33         old_index = ALPHABET_DICT[char]
34         new_index = (old_index + (N - key)) % N
35         result.append(ALPHABET[new_index])
36     return "".join(result)
37
38
```

Рисунок 1 — Реализация функций шифрования и дешифрования шифра Цезаря



```
71
72 if __name__ == "__main__":
73     sample = "Криптография изучает методы защиты информации"
74     key = 3
75     enc = caesar_encrypt(sample, key)
76     print("Шифртекст:", enc)
77     print("Расшифровка:", caesar_decrypt(enc, key))
78
79     # with open("challenge_caesar.txt", encoding="utf-8") as f:
80     #     cipher = f.read()
81     # found_key, text = crack_caesar(cipher)
82     # print("Найденный ключ:", found_key)
83     # print("Расшифровка:", text)
84
```

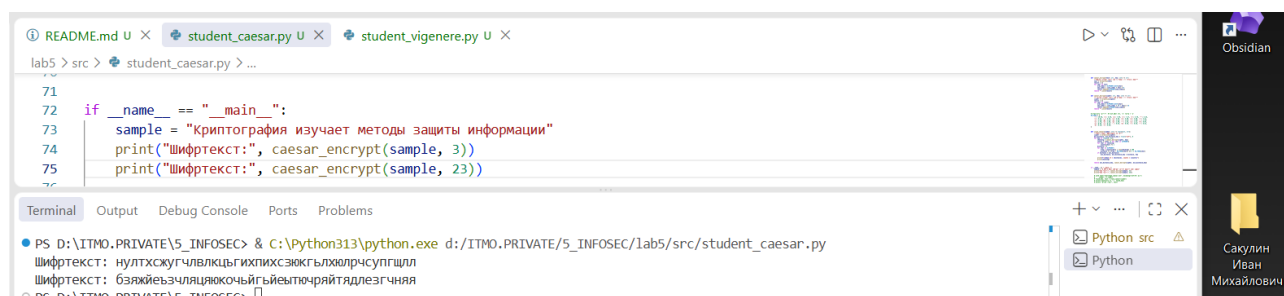
Terminal Output:

```
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src> & C:\Python313\python.exe d:\ITMO.PRIVATE\5_INFOSSEC\lab5\src\student_caesar.py
Шифртекст: нултхсжугчлвлцкьгхпхксзюкгълолрчсупгцлл
Расшифровка: криптографияизучаетметодызащитыинформации
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src>
```

Рисунок 2 — Проверка шифрования и дешифрования

1.2 Лавинность подстановки и количество ключей

Для демонстрации лавинности подстановки одна и та же фраза была зашифрована двумя разными ключами: 3 и 23. Результаты представлены на рисунке 3. Шифртексты различаются полностью, что показывает чувствительность результата к изменению ключа.



```
1 README.md x student_caesar.py x student_vigenere.py x
lab5 > src > student_caesar.py > ...

71
72 if __name__ == "__main__":
73     sample = "Криптография изучает методы защиты информации"
74     print("Шифртекст:", caesar_encrypt(sample, 3))
75     print("Шифртекст:", caesar_encrypt(sample, 23))
76

Terminal Output Debug Console Ports Problems
PS D:\ITMO.PRIVATE\5_INFOSEC> & C:\Python313\python.exe d:\ITMO.PRIVATE\5_INFOSEC\lab5/src/student_caesar.py
Шифртекст: нултхсжугчвлкцггхпххсзжкгьлхлрчсупгшл
Шифртекст: бзжйеъзчляцкочъгъейтчряйтдлзгчняя
PS D:\ITMO.PRIVATE\5_INFOSEC> |
```

Рисунок 3 — Шифрование одной фразы разными ключами

В 31-буквенном алфавите число возможных ключей шифра Цезаря равно 31 (сдвиги от 0 до 30). Это делает шифр тривиально взламываемым простым перебором: даже вручную можно проверить все варианты, а на компьютере перебор занимает доли секунды.

1.3 Взлом шифра Цезаря частотным анализом

Реализована функция `crack_caesar`, которая перебирает все возможные сдвиги и для каждого вычисляет значение хи-квадрат между частотным распределением расшифрованного текста и эталонными частотами русского языка. Сдвиг, дающий минимальное значение хи-квадрат, считается наиболее вероятным ключом.

На рисунке 4 показан код реализации этой функции, а на рисунке 5 — результат её работы на файле `challenge_caesar.txt`.



```
1  README.md x student_caesar.py x student_vigenere.py x
lab5 > src > student_caesar.py > crack_caesar

39 # Эталонные частоты русских букв (%), 'ё' влита в 'е'
40 RU_FREQ = {
41     'а': 10.98, 'б': 8.49, 'в': 7.99, 'г': 7.37, 'д': 6.70, 'е': 6.26,
42     'ж': 5.47, 'з': 4.75, 'и': 4.54, 'к': 3.49, 'л': 3.21,
43     'м': 2.98, 'н': 2.81, 'о': 2.62, 'п': 2.01, 'р': 1.90, 'с': 1.74,
44     'т': 1.70, 'у': 1.65, 'ф': 1.59, 'х': 1.44, 'ц': 1.21, 'ч': 0.97,
45     'ш': 0.94, 'щ': 0.73, 'ю': 0.64, 'я': 0.48, 'ь': 0.36, 'э': 0.32,
46     'ё': 0.26, 'ъ': 0.04,
47 }
48
49
50 def crack_caesar(cipher: str) -> tuple[int, str]:
51     """Взлом Цезаря БЕЗ знания ключа"""
52     cipher = normalize(cipher)
53     min_distance, min_distance_key = float("inf"), 0
54     for key in range(0, 31):
55         checking = caesar_decrypt(cipher, key)
56         counts = {char: 0 for char in ALPHABET}
57         for char in checking:
58             counts[char] += 1
59         distance = 0
60         for char in ALPHABET:
61             freq = counts[char] / len(checking) * 100
62             distance += (freq - RU_FREQ[char]) ** 2 / RU_FREQ[char]
63         if distance < min_distance:
64             min_distance, min_distance_key = distance, key
65     return min_distance_key, caesar_decrypt(cipher, min_distance_key)
```

Рисунок 4 — Реализация функции взлома шифра Цезаря



```
65 return min_distance_key, caesar_decrypt(cipher, min_distance_key)
66
67
68 if __name__ == "__main__":
69     with open("challenge_caesar.txt", encoding="utf-8") as f:
70         cipher = f.read()
71     found_key, text = crack_caesar(cipher)
72     print("Найденный ключ:", found_key)
73     print("Расшифровка:", text)
74
```

Terminal Output:

```
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src> & C:\Python313\python.exe d:/ITMO.PRIVATE/5_INFOSSEC/lab5/src/student_caesar.py
Найденный ключ: 14
Расшифровка: информация безопасности строится на трех китах: конфиденциальность, целостность, доступность. Нарушение любого из этих свойств приводит к ущербу классической шифры.
Интересны прежде всего о каучебный материал, показывающий почему стойкость криптосистемы не должна зависеть от секретности алгоритма
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src>
```

Рисунок 5 — Результат взлома: найденный ключ и расшифрованный текст

Текст расшифрован подобранным ключом, равным 14.

1.4 Почему частотный анализ работает

Частотный анализ основан на том, что в любом естественном языке частоты появления букв примерно постоянны для достаточно длинных текстов. В моноалфавитном шифре каждая буква исходного текста заменяется на одну и ту же букву шифртекста. Поэтому частотное распределение букв в шифртексте является просто сдвинутым распределением исходного языка.

2 ШИФР ВИЖЕНЕРА

2.1 Реализация шифра Виженера

В файле `student_vigenere.py` реализованы функции `vigenere_encrypt` и `vigenere_decrypt`. На рисунке 6 показан код реализованных функций, а на рисунке 7 — проверка на паре «текст + ключевое слово»: шифрование и последующая расшифровка дают исходный текст.



```
9 def vigenere_encrypt(text: str, keyword: str) -> str:
10     """Зашифровывает текст шифром Виженера ключевым словом keyword"""
11     text = normalize(text)
12     keyword = normalize(keyword)
13     result = []
14     for i, char in enumerate(text):
15         old_index = ALPHABET_DICT[char]
16         shift = ALPHABET_DICT[keyword[i % len(keyword)]]
17         new_index = (old_index + shift) % N
18         result.append(ALPHABET[new_index])
19     return "".join(result)
20
21
22 def vigenere_decrypt(cipher, keyword):
23     """Расшифровывает текст шифра Виженера ключевым словом keyword"""
24     cipher = normalize(cipher)
25     keyword = normalize(keyword)
26     result = []
27     for i, char in enumerate(cipher):
28         old_index = ALPHABET_DICT[char]
29         shift = ALPHABET_DICT[keyword[i % len(keyword)]]
30         new_index = (old_index + (N - shift)) % N
31         result.append(ALPHABET[new_index])
32     return "".join(result)
33
```

Рисунок 6 — Реализация функций для шифра Виженера



```
73 if __name__ == "__main__":
74     sample = "Принцип Керкгоффса лежит в основе современной криптографии"
75     kw = "ключ"
76     enc = vigenere_encrypt(sample, kw)
77     print("Шифртекст:", enc)
78     print("Расшифровка:", vigenere_decrypt(enc, kw))
79
```

Terminal Output:

```
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src> & C:\Python313\python.exe d:\ITMO.PRIVATE\5_INFOSSEC\lab5\src\student_vigenere.py
Шифртекст: шьждаунбпыишятикгэзаеышпъмьдркышмафьюкышбзакяя
Расшифровка: принципкеркгоффса лежит в основе современной криптографии
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src>
```

Рисунок 7 — Проверка шифрования и дешифрования шифром Виженера

2.2 Отличие от шифра Цезаря

В шифре Виженера одна и та же буква в разных позициях текста может превращаться в разные буквы шифртекста, поскольку сдвиг зависит от позиции (определяется соответствующим символом ключа). Это продемонстрировано на рисунке 8: буква **а** в разных позициях зашифрована по-разному.



Рисунок 8 — Разные позиции буквы в шифре Виженера

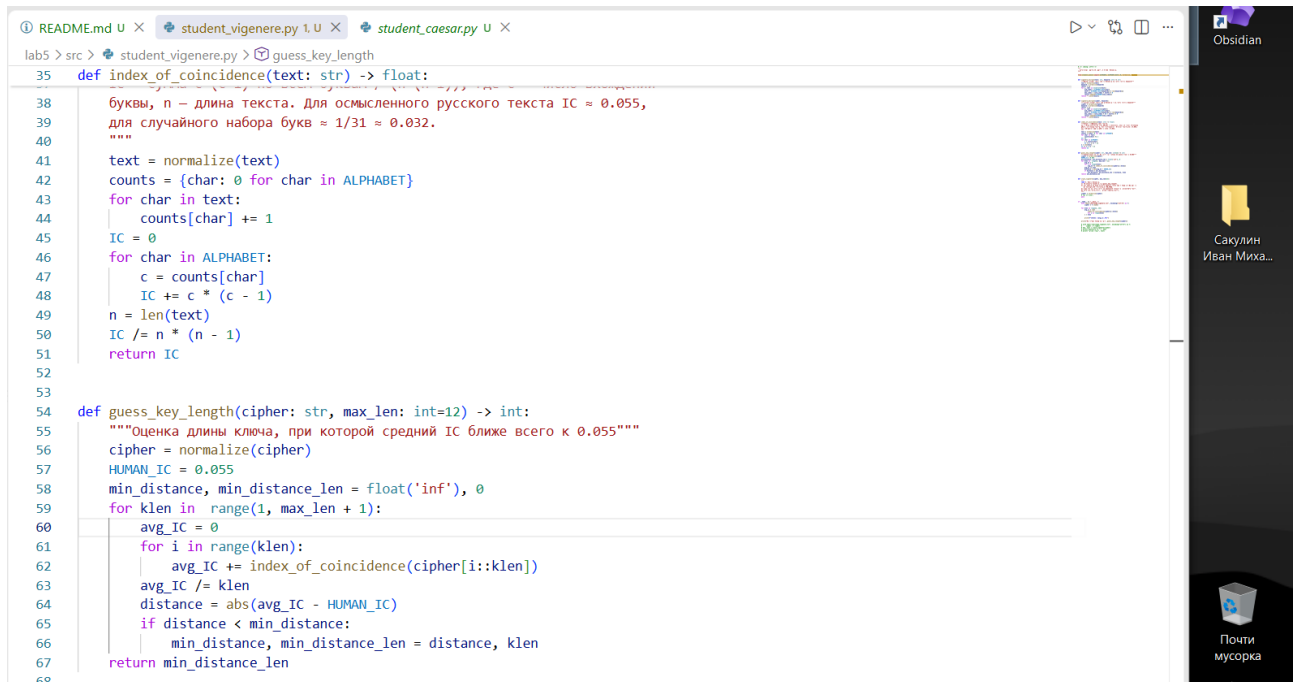
В результате частоты по всему шифртексту размазываются, и перебор всех сдвигов, как у Цезаря, ничего не даст. Сначала надо определить длину ключа, разбить шифртекст на столбцы, и только потом каждый столбец ломать как обычный Цезарь по частотам.

2.3 Оценка длины ключа через индекс совпадений

Реализованы функции `index_of_coincidence` и `guess_key_length`. На рисунке 9 показан код этих функций, а на рисунке 10 — таблица индексов совпадений для длин ключа от 1 до 12, полученная при запуске на файле `challenge_vigenere.txt`.

Наиболее вероятная длина ключа — 8, так как для неё индекс совпадений (0.05537) значительно выше, чем для соседних значений. Также высокие значения наблюдаются для длин 4 и 12, которые кратны 8. Это объясняется тем, что при правильной длине ключа (или кратной ей) шифртекст разбивается на потоки, каждый из которых представляет собой моноалфавитную

подстановку, и частотное распределение внутри потока приближается к естественному, что даёт более высокий индекс совпадений.



```
1  README.md  student_vigenere.py 1.0  student_caesar.py
lab5 > src > student_vigenere.py > guess_key_length
35 def index_of_coincidence(text: str) -> float:
36     """
37     буквы, n – длина текста. Для осмысленного русского текста IC ≈ 0.055,
38     для случайного набора букв ≈ 1/31 ≈ 0.032.
39     """
40     text = normalize(text)
41     counts = {char: 0 for char in ALPHABET}
42     for char in text:
43         counts[char] += 1
44     IC = 0
45     for char in ALPHABET:
46         c = counts[char]
47         IC += c * (c - 1)
48     n = len(text)
49     IC /= n * (n - 1)
50     return IC
51
52
53
54 def guess_key_length(cipher: str, max_len: int=12) -> int:
55     """Оценка длины ключа, при которой средний IC ближе всего к 0.055"""
56     cipher = normalize(cipher)
57     HUMAN_IC = 0.055
58     min_distance, min_distance_len = float('inf'), 0
59     for klen in range(1, max_len + 1):
60         avg_IC = 0
61         for i in range(klen):
62             avg_IC += index_of_coincidence(cipher[i::klen])
63         avg_IC /= klen
64         distance = abs(avg_IC - HUMAN_IC)
65         if distance < min_distance:
66             min_distance, min_distance_len = distance, klen
67     return min_distance_len
68
```

Рисунок 9 — Реализация функций для вычисления индекса совпадений



```
1  README.md  student_vigenere.py 1.0
lab5 > src > student_vigenere.py > guess_key_length
83
84 if __name__ == "__main__":
85     with open("challenge_vigenere.txt", encoding="utf-8") as f:
86         cipher = f.read()
87
88     for klen in range(1, 13):
89         avg_ic = sum(
90             index_of_coincidence(cipher[i::klen])
91             for i in range(klen)
92         ) / klen
93
94         print(f"{klen}: {avg_ic:.5f}")
95
96     print("Вероятная длина ключа:", guess_key_length(cipher))
97
```

```
Terminal  Output  Debug Console  Ports  Problems
Python - src
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src> & C:\Python313\python.exe d:/ITMO.PRIVATE/5_INFOSSEC/lab5/src/student_vigenere.py
1: 0.03438
2: 0.04237
3: 0.03483
4: 0.05772
5: 0.03591
6: 0.04281
7: 0.03151
8: 0.05537
9: 0.03315
10: 0.04732
11: 0.03259
12: 0.05761
Вероятная длина ключа: 8
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src>
```

Рисунок 10 — Индексы совпадений для длин ключа от 1 до 12

2.4 Полный взлом шифра Виженера

Реализована функция `crack_vigenere`, которая:

- 1) разбивает шифртекст на потоки по найденной длине ключа;
- 2) для каждого потока выполняет частотный анализ (как для шифра Цезаря) и определяет наиболее вероятный сдвиг;
- 3) собирает из найденных сдвигов ключевое слово;
- 4) расшифровывает исходный текст.

На рисунке 11 показан код функции взлома, а на рисунке 12 приведён итоговый результат — восстановленное ключевое слово и расшифрованный текст.



```
69
70 def crack_vigenere(cipher: str, max_len: int=12) -> tuple[str, str]:
71     """Взлом Виженера"""
72     cipher = normalize(cipher)
73     klen = guess_key_length(cipher)
74     key = ""
75     for i in range(klen):
76         column = cipher[i::klen]
77         min_distance, best_key = float('inf'), 0
78         for shift in range(31):
79             checking = caesar_decrypt(column, shift)
80             counts = {char: 0 for char in ALPHABET}
81             for char in checking:
82                 counts[char] += 1
83             distance = 0
84             for char in ALPHABET:
85                 freq = counts[char] / len(checking) * 100
86                 distance += (freq - RU_FREQ[char]) ** 2 / RU_FREQ[char]
87             if distance < min_distance:
88                 min_distance = distance
89                 best_key = shift
90             key += ALPHABET[best_key]
91     decrypted = vigenere_decrypt(cipher, key)
92     return key, decrypted
93
```

Рисунок 11 — Реализация итоговой функции взлома шифра Виженера



```
93
94 if __name__ == "__main__":
95     with open("challenge_vigenere.txt", encoding="utf-8") as f:
96         cipher = f.read()
97         key, text = crack_vigenere(cipher)
98         print("Найденный ключ:", key)
99         print("Расшифровка:", text)
100
```

Terminal Output:

```
PS D:\ITMO.PRIVATE\5_INFOSSEC\lab5\src> & C:\Python313\python.exe d:\ITMO.PRIVATE\5_INFOSSEC\lab5\src\student_vigenere.py
Найденный ключ: ключключ
Расшифровка: настоящая криптографическая стойкость достигается только тогда, когда секретом является исключительно ключ, а сам алгоритм открыт. Общеизвестен тот принцип формулировки логической задачи, что если девять из десяти возможных вариантов решения являются неверными, то оставшийся вариант является верным. В современной криптографии открытость алгоритма позволяет сообществу специалистов находить уязвимости в алгоритме, а также создавать инструменты для взлома. Поэтому алгоритм должен быть настолько совершенным, насколько это возможно, чтобы обеспечить безопасность информации.
```

Рисунок 12 — Результат взлома: ключ и расшифрованный текст

ЗАКЛЮЧЕНИЕ

В ходе практической работы были реализованы и исследованы шифры Цезаря и Виженера. Для шифра Цезаря выполнено шифрование и дешифрование текста, а также реализован взлом методом перебора с использованием критерия хи-квадрат. Для шифра Виженера реализованы функции шифрования и дешифрования, определение длины ключа по индексу совпадений и восстановление ключа посредством частотного анализа отдельных потоков.

Принцип Керкгоффа заключается в том, что стойкость криптографической системы должна определяться секретностью ключа, а не алгоритма. В рамках работы алгоритмы и их реализации были известны заранее, при этом ключи шифртекстов не были известны. Несмотря на это, ключи удалось восстановить с использованием статистических методов. Полученный результат подтверждает применимость данного принципа к рассмотренным классическим шифрам.

Эффективность частотного анализа зависит от длины шифртекста. При увеличении длины сообщения статистическое распределение букв приближается к характерному для исходного языка, что повышает точность определения ключа. При коротком шифртексте статистические отклонения становятся существенными, поэтому результат частотного анализа может быть неточным.

Стойкость шифра Виженера зависит от длины ключа. Увеличение длины ключа увеличивает количество возможных вариантов ключа и уменьшает длину отдельных потоков, что усложняет частотный анализ. При длине ключа, равной длине сообщения, и однократном использовании случайного ключа шифр Виженера соответствует одноразовому блокноту и обеспечивает теоретическую криптографическую стойкость.